# Robot Agentic

## The code explained

Software anomaly detection on URSim: what each part does, how it was built, which libraries and why, how big it is, and what is still missing.

Branch `refactor/software-detection`, 9 commits on top of `main`

State of the code on 1 November 2026

Written to be read on a phone or a tablet: A5 pages, one idea per section. Words in `typewriter` are names in the code; the glossary at the end explains the robot words.

# Contents

**1 The project in one page** **3**

**2 The big picture** **3**

**3 How a run goes, step by step** **5**

3.1 `robot-agentic check "the arm struggles"` . . . . . 5

3.2 `robot-agentic loop` . . . . . 5

3.3 `robot-agentic health -api` . . . . . 5

**4 Languages, libraries and tools, and why** **6**

4.1 Inside the program . . . . . 6

4.2 Python's own library (no install) . . . . . 6

4.3 For development . . . . . 6

4.4 Outside the program . . . . . 7

4.5 What was *not* used, on purpose . . . . . 7

**5 The core: `src/robot_agentic/core/`** **7**

**6 The URSim connector: `connectors/ursim.py`** **9**

6.1 The robot's doors (network ports) . . . . . 9

6.2 Reading the signals . . . . . 9

6.3 Reading the fault codes . . . . . 10

6.4 Reading the logs . . . . . 10

6.5 The black boxes . . . . . 10

6.6 The tests offered to Claude . . . . . 11

**7 The agent: `src/robot_agentic/agent/`** **11**

**8 Around the agent** **13**

**9 Outside the package: tools, benchmarks, tests** **14**

9.1 Fault injection: `tools/fault_injection/` . . . . . 14
9.2 Benchmarks: `benchmarks/run.py` . . . . . 15
9.3 Tests: `tests/` . . . . . 15

**10 How it was developed** **15**
10.1 The code principles (CONTRIBUTING.md) . . . . . 15
10.2 The history of this branch . . . . . 16
10.3 Bugs found by testing on the real simulator . . . . . 16

**11 How big is it?** **17**
11.1 Lines, by part . . . . . 17
11.2 The agent package, file by file . . . . . 17
11.3 Compared with `main` . . . . . 18

**12 What is missing** **18**
12.1 1. The detection rules on the signals . . . . . 18
12.2 2. The expected current in the live stream . . . . . 18
12.3 3. A “stream health” rule (proposed rule g) . . . . . 18
12.4 4. Benchmarks: never run, and incomplete . . . . . 18
12.5 5. Smaller gaps in the code . . . . . 19
12.6 6. Planned for later (summary.md, section 6) . . . . . 19

**13 Glossary** **19**

# 1 The project in one page

**The goal.** When an industrial robot breaks down, a technician looks at what the robot reports, makes hypotheses, tests them, and finds the cause. Robot Agentic does the same thing with an AI agent (Claude). It reads the robot's **four data sources**:

1. the **signals**: joint positions, speeds, currents, torques, temperatures, 500 times per second;
2. the **logs**: the text lines the robot's programs write;
3. the **fault codes**: the official error numbers of the controller (e.g. `C209A0`);
4. the **documentation**: the official manual pages.

It then runs small, slow, operator-approved tests on the robot, and hands in a report: *the root cause, the evidence, the repair.*

**Two independent approaches.** You defined two ways of detecting anomalies, that never mix in one investigation:

- **software anomaly detection**: uses only what the robot's software reports (the four sources above). *This is the only one in the code today.*
- **physical anomaly detection**: uses our own sensors (microphone, accelerometer, current clamp) on the machine, even a closed one. Planned for later (fan first, then the RoArm-M2-S).

**The test bench.** We do not have a real Universal Robots arm. We use **URSim**, the official simulator from Universal Robots: the real controller software, running in a Docker container on the laptop. To the code, URSim looks exactly like a real UR3e robot: same network ports, same binary formats, same logs.

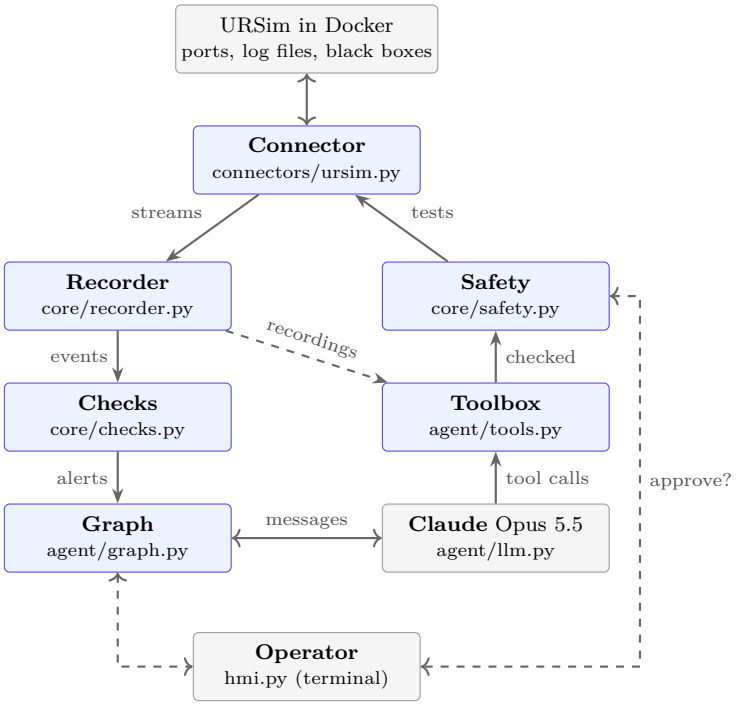
**Two ways to use the agent.**

- `robot-agentic check`: a spot check, on demand ("is the robot OK?"), or a question ("the arm struggles when lifting").
- `robot-agentic loop`: continuous monitoring. Simple rules (no AI, no cost) watch the logs and fault codes; when something looks wrong, the operator is asked whether to start an investigation.

And `robot-agentic health` checks that everything is connected.

# 2 The big picture

The code is cut into four layers. Each layer only knows the one below it.




1. **The connector** is the only part that knows URSim. It turns raw bytes and text into four simple Python types: `Sample`, `LogLine`, `FaultCode` and `Recording`.
2. **The core** (no AI, no robot brand): the **recorder** keeps the last two minutes of every stream in memory; the **checks** raise alerts; the **safety** layer refuses or asks before any movement.
3. **The agent**: the **graph** drives an investigation, the **toolbox** runs what Claude asks for, **context** turns recordings into plots and tables, **llm** talks to Claude.
4. **Around it**: the command line, the terminal interface, the settings, the documentation search; and outside the package, the fault-injection tools, the benchmarks and the tests.

**Why this cut?** Plugging in a new machine (the RoArm-M2-S, a car through OBD-II…) means writing one new connector file. Nothing else changes, because the rest of the code only knows the four simple types and the `Connector` contract.

# 3 How a run goes, step by step

## 3.1 `robot-agentic check "the arm struggles"`

1. `__main__.py` reads the settings (`.env`), creates the URSim connector, starts the recorder (three background threads read signals, logs and fault codes), and builds the toolbox.
2. The graph starts at the `investigate` step. It sends Claude a first message with what it would ask for anyway: the request, the robot state, the fault codes and the log warnings of the last two minutes.
3. Claude thinks, then calls tools: e.g. `search_docs("C209")`, then `joint_test(joint=1, delta_deg=3)`.
4. For a test, the safety layer checks the bounds (±5° max), asks the operator "*The agent wants to run joint_test(joint=1, delta_deg=3). Allow?*" and the PIN once.
5. The connector sends a tiny URScript program to the robot; the recorder cuts out everything that happened from 1 s before to 1.5 s after the test.
6. Claude receives three things: a text summary (stream cuts, fault codes, log warnings), a plot of the channels that moved most, and the full values as a table.
7. This repeats (25 turns maximum) until Claude calls `submit_report`. The report is saved as JSON in `data/reports/` and shown in the terminal, with the cost of the calls.

## 3.2 `robot-agentic loop`

1. The graph starts at `watch`: it reads every new log line and fault code. No Claude call, no cost.
2. The `EventWatcher` rules decide if an event is an alert (a fault code, a dangerous phrase like "protective stop", or an ERROR line).
3. The operator is asked: "*Anomaly: fault code C209A0. Start a root-cause analysis?*"
4. If yes: the same investigation as above, then monitoring resumes. Events that arrived during the investigation are dropped (they were mostly caused by the agent's own tests).

## 3.3 `robot-agentic health -api`

One line per check, ✓ or ×: Dashboard reachable and robot RUNNING, Real-Time stream with its rate and channel count, log stream, fault stream, documents

found, and (with `-api`) access to the Claude model, through a free API request.

# 4 Languages, libraries and tools, and why

## 4.1 Inside the program

| | |
|---|---|
| **Python 3.11+** | The language of robotics and AI tooling; every library below exists for it; it runs on the Raspberry Pi. |
| **anthropic (2.3)** | Official Claude SDK. Handles retries, authentication (API key or `ant auth login`), and the newer features we use: adaptive thinking, effort, prompt caching, server fallback. |
| **LangGraph (1.2)** | Describes the investigation as a small graph (watch → ask → investigate → report). You chose to keep it: the flow is visible, and adding a step later (e.g. "ask a human expert") is one node. |
| **numpy (2.5)** | The ring buffer of signals (one big array), resampling, cut detection. 500 samples/s × 63 channels is too much for plain Python objects on a Pi. |
| **matplotlib (3.11)** | Draws the plot sent to Claude after each test (PNG, in memory). |
| **pypdf (6.21)** | Reads PDF manuals to index them, if PDFs are added to `manuals/`. |
| **python-dotenv** | Reads the `.env` file (API key, robot address, PIN). |
| **PyYAML** | Reads the scenarios, the catalogue sources (and later the detection thresholds). |

## 4.2 Python's own library (no install)

`socket` and `struct` (talk to the robot's ports and decode its binary packets), `subprocess` (run `docker` commands), `threading` and `queue` (the three reader threads), `dataclasses` (the simple types), `zipfile` (open black boxes), `gzip`/`json` (save recordings), `logging`, `argparse` (command line).

## 4.3 For development

| | |
|---|---|
| **pytest (9.1)** | 24 automatic tests, run in seconds, with no robot, no network and no cost. |
| **ruff (0.17)** | Checks style, imports, *and* the two rules of CONTRIBUTING.md that a tool can check: type annotations and docstrings everywhere. |
| **setuptools** | Packages the project so that `pip install -e .` creates the `robot-agentic` command. |

## 4.4 Outside the program

| | |
|---|---|
| **Docker** | Runs URSim (image `universalrobots/ursim_e-series`, version 5.26.1). Also the tool to provoke faults: pause, CPU limit, network disconnect. |
| **Alpine helper image** | 19 MB image with `tc` and `iptables`, joined to URSim's network to add delay, loss or block a port. URSim's image has neither tool. |
| **udisks** | Attaches the 512 MB disk file URSim runs on, without admin rights, so "disk full" never fills the laptop. |
| **LaTeX** | The fault catalogue and this guide. |

## 4.5 What was *not* used, on purpose

- **pydantic** (validated models): too slow at 500 samples/s, and each connector already parses strictly. Plain dataclasses instead.
- **pandas**: heavy for a Raspberry Pi; numpy is enough.
- **A vector database / embeddings** for the documentation: fault codes and part names are exact words, which classic keyword ranking (BM25) handles better, with zero dependency and zero cost.
- **LangChain** or an agent framework around Claude: the loop is 60 lines and fully visible; LangGraph is used only for the flow.
- **The UR RTDE library**: we read the Real-Time port directly with `struct`, which needs no extra package and works the same on a Pi.

# 5 The core: `src/robot_agentic/core/`

These five files know nothing about Universal Robots or Claude.

**`events.py`** 123 lines

The shared vocabulary. Every other file speaks in these types:

- `Sample`: one instant of the signal stream, a time `t` and a dictionary `{"joint_2_current_actual": 0.41, ...}`.
- `LogLine` (time, level, source, text) and `FaultCode` (time, code, message, level).
- `Level` (DEBUG to FATAL) and `Severity` (warning, critical).
- `Alert`: what wakes the agent up, with its evidence.
- `Param` and `ToolSpec`: how a connector describes a test, e.g. "`joint`: integer 0–5, `delta_deg`: number −5 to 5, moves the machine".

The bounds of a test are written *once*, in its `Param`. They are shown to Claude (as `minimum`/`maximum` in the tool's JSON schema) *and* enforced by the safety layer, so the two can never disagree.

### `interfaces.py` 69 lines

The `Connector` contract: what every machine must offer. Three streams (`signals()`, `logs()`, `faults()`), `docs()`, `tools()`, `call_tool()`, `stop()`, `health()`, and an optional `snapshot()` (current state). It is an abstract base class: Python refuses to create a connector that forgets one method.

### `recorder.py` 305 lines

The memory of the system.

- `Recorder` starts one thread per stream. If a stream breaks (robot off, network cut), the thread waits one second and reconnects; the cause is kept in `errors`, because a lost stream is itself a clue.
- The signals go into a **ring buffer**: one numpy array of 60 000 rows (120 s at 500 Hz) where the newest row overwrites the oldest. 120 s on 63 channels is about 60 MB instead of more than 200 MB with one Python object per sample.
- `window(t0, t1)` cuts any time slice out of the buffers into a `Recording`; `record(action)` runs a test and keeps 1 s before and 1.5 s after it. Because the buffer is always filling, the start of a test is never missed.
- `subscribe()` gives a queue that receives every new log line and fault code: this is what the loop mode watches.
- `stop()` ends the threads cleanly (added after a bug: leftover threads polluted the fault captures).
- `Recording` can `resample` (average down to fewer points per second, which keeps short peaks), `save` to `.json.gz` and `load`.

### `checks.py` 87 lines

The rules without AI.

- `EventWatcher.feed(event)`: a fault code always gives an alert; a log line containing "protective stop", "emergency stop", "lost communication", "connection lost", "fault" (as a whole word, so "default" does not match), "collision" or "overcurrent" gives a critical alert; any ERROR line gives a warning. The same reason is reported at most once every 30 s, so a repeating failure does not flood the operator.

- `stream_gaps(recording)`: the holes longer than 0.1 s in the signal stream, including a stream that stops for good before the end of the window.

This is where the signal rules (b, e, a, c) will go.

**`safety.py`** 76 lines

Every action Claude asks for passes here first. `SafetyPolicy.check`:

1. every argument must be present, be a number, and be inside its range; otherwise it is refused (never silently clamped) and Claude is told why;
2. if the action moves the robot: the operator is asked, *every time*;
3. if a PIN is configured: it is asked before the first movement of each investigation.

Even if Claude makes a mistake or is manipulated by a strange log line, the robot cannot leave limits fixed in advance by humans. Claude only sees "REFUSED: . . . " and has to adapt.

# 6 The URSim connector: `connectors/ursim.py`

**`ursim.py`** 672 lines

The biggest file, on purpose: it is the only place that knows the Universal Robots formats (your decision: one file). It reads in order, top to bottom.

## 6.1 The robot's doors (network ports)

| Port | Name | What we do with it |
|---|---|---|
| 30003 | Real-Time | Read the signals: one binary packet every 2 ms (500 Hz). |
| 30001 | Primary | Read the fault codes ("Robot Message" packets). |
| 30002 | Secondary | Send URScript programs (the robot's language) for the tests. |
| 29999 | Dashboard | Text commands: robot mode, safety status, power on, unlock a protective stop. |

## 6.2 Reading the signals

`_packets` splits the byte stream into packets (each starts with its own size). `parse_realtime` reads, at fixed positions in the packet (the table `REALTIME_FIELDS`, from the official "Remote Control via TCP/IP" document), 16 groups of values: target and actual joint positions, speeds, currents, target torques, hand pose and force, motor temperatures, robot mode, safety mode,

accelerometer, speed scaling, voltages. Each becomes a named channel such as `joint_1_current_actual`.

## 6.3 Reading the fault codes

`parse_primary` decodes three message types: error codes, "key" messages at level ERROR or above, and runtime exceptions of a program. A code is written as on the robot's tablet: `C209A0` (code 209, argument 0). Messages replayed by the controller during the first second of a connection are skipped (they are old).

## 6.4 Reading the logs

There are three log files inside the container: the controller's (`URControl.log`), the user interface's (`polyscope.log`) and the teach pendant's event list (`log_history.txt`, what a technician sees on the tablet). `logs()` runs `tail -F` on the three through `docker exec`, the way one would read them over SSH on a real controller.

`parse_log_line` recognises the file from the line's date format, removes colour codes and NUL characters, extracts the level, keeps multi-line Java messages together, and demotes the noise of client connections (including ours) to DEBUG so it never raises an alert.

A trap found during the study: killing our `docker exec` did not stop the `tail` inside the container. The fix: `tail` runs in the background of a small shell that waits on our input; when we stop, the input closes and the shell kills `tail`.

## 6.5 The black boxes

On every error, the robot writes a "flight report": a zip with about 30 s of its own data at 500 Hz (181 columns, more than the live stream: *target current*, deviation ratios, payload), and its logs. `read_flight_report` turns it into a normal `Recording`, renaming columns to the live-stream names so Claude compares like with like, and aligning the time on the report's "Reporting Time". The tool `get_flight_report(index)` copies one out of the container; an unreadable one (e.g. written on a full disk) gives a clear message instead of a crash.

### 6.6 The tests offered to Claude

| | |
|---|---|
| `joint_test` | One joint turns ±5° max at 10 % speed, and comes back. |
| `move_hand` | The hand moves ±2 cm max in a straight line, and comes back. |
| `add_load` | Declares up to 1.5 kg more payload, repeats a shoulder test, restores the original payload (even after an interruption, through `recover_robot`). |
| `overall_test` | Each joint in turn, 1–5°: compare the joints with each other. |
| `recover_robot` | Close popups, unlock a protective stop, power on, release brakes. |
| `get_flight_report` | Open a black box. Does not move the robot. |

Each test is written in URScript (`send_script`), started, and its end is detected by asking the Dashboard "running?" every 0.1 s (60 s timeout, then an emergency `stopj`).

**Why does `add_load` read and restore the payload?** A wrong payload declaration may be the very fault under investigation. Overwriting it with a fixed value would erase the evidence.

**`fake.py`** 136 lines

A fake six-joint arm in pure Python (100 Hz), used by the tests: realistic currents (gravity + speed × friction), one `joint_test`, and `inject("friction")` or `inject("fault")` to simulate a problem. Fast, free and reproducible.

## 7 The agent: `src/robot_agentic/agent/`

**`graph.py`** 251 lines

The course of an investigation, as a LangGraph graph with four steps:

$$\texttt{watch} \xrightarrow{\text{alert}} \texttt{ask_approval} \xrightarrow{\text{yes}} \texttt{investigate} \rightarrow \texttt{report}$$

`investigate` is the heart: the conversation with Claude. At each turn Claude answers with thinking, text and tool calls; the tools run; their results are added; and again, until `submit_report` or 25 turns. If Claude answers without any tool, it is reminded to conclude. A failing tool returns its error to Claude instead of crashing the investigation.

**The conversation is only ever extended, never edited.** This is what makes prompt caching work (each turn only pays full price for its new part) and keeps Claude's reasoning from one turn to the next.

`run_check` and `run_loop` are the two modes of section 3.

**`tools.py`** 239 lines

The `Toolbox`: the list of tools Claude can call and the code that runs them.

- Common tools (any machine): `get_status`, `get_recent_logs`, `get_recent_faults`, `search_docs`, `get_recording` (a recording as a table, or `"latest:N"` for the last N seconds), `plot_recording`, `submit_report`.
- Machine tools come from the connector and always go through the safety layer.
- `submit_report` uses a strict JSON schema: root cause, confidence, evidence, repair, references, tests done. Claude cannot hand in a report with a field missing.
- `describe_test`: what Claude gets after each test (summary + plot + full table).

**`context.py`** 113 lines

How a recording is shown to Claude without exceeding the budget (`RECORDING_TOKEN_BUDGET`, 40 000 tokens by default).

- `moving_channels`: the channels that vary, ranked by how much (range divided by size, so radians, amperes and degrees can be compared). Constant channels are left out.
- `plot`: up to 8 channels, one per row, as a PNG.
- `csv_within_budget`: the table at the finest rate that fits; the rate is halved until it fits. A 5 s test at 500 Hz on 63 channels is 160 000 numbers, far too many.

**Why a plot *and* a table?** Research (Zhou & Yu, ICLR 2025) shows LLMs spot anomalies in time series better on an image than in a list of numbers. The table keeps the exact values for checking. You chose to give both.

**`llm.py`** 141 lines

`ClaudeClient.create`: one call to Claude Opus 5.5 with

- **adaptive thinking** (Claude decides how much to think; a summary is shown in grey);
- **effort high** (your choice: a diagnosis needs rigour);
- **prompt caching**: a cache point on the system prompt plus automatic caching of the conversation, so a 20-turn investigation does not pay 20 times for the same history;

- **server fallback**: if the model declines, another model answers, and a warning says so.

Each call is appended to `data/llm_calls.jsonl` with its tokens, cost and duration. `cost_usd` computes the price from the public rates ($4 / $20 per million input / output tokens for Opus 5.5).

**`prompts/system.md`** 27 lines

The agent's instructions, in plain English: what it has (four sources, tests, black boxes), the method (facts → ranked hypotheses → the smallest test that separates them → conclude), and the rules (no repeating a refused test, check one sensor against another, a communication failure is a root cause in itself, quote every claim).

# 8 Around the agent

**`__main__.py`** 141 lines

The `robot-agentic` command: `build_runtime` assembles everything (also used by the benchmarks), `health` runs the checklist, `main` reads the command line. Ctrl+C stops the robot's movement before exiting, and the total Claude cost is always printed.

**`config.py`** 82 lines

`Settings`: robot address, container name, model, token budget, PIN, log level, folders. Each field is read from the environment variable of the same name in capitals (`ROBOT_HOST`…), or from `.env`. `setup_logging`: everything goes to `data/robot_agentic.log`, only warnings to the terminal.

**`hmi.py`** 77 lines

The terminal interface (the only file allowed to `print`): alerts in red, questions `[y/N]`, the PIN typed without being shown, tool calls in blue, the report. `-yes` answers yes automatically (simulator and benchmarks only).

**`docs.py`** 150 lines

The documentation search.

- `build` cuts each document (PDF, HTML, Markdown, text) into passages of about 1 500 characters. A Markdown heading always starts a new passage, and long sections repeat their heading and source address, so every passage can be quoted with its origin. The result is cached and rebuilt only when a file changes.
- `DocSearch` ranks passages with **BM25**, the formula of classic search engines:

rare words that appear often in a short passage score high. A code like `C204A3` is also searched as `C204`.

Today's documentation: the 225 official UR error code pages (software 5.26), downloaded by `tools/fetch_ur_docs.py`, each with its web address.

# 9 Outside the package: tools, benchmarks, tests

## 9.1 Fault injection: `tools/fault_injection/`

**`inject.py`** 233 lines

`URSimFaults`: how to provoke and repair each of the 13 faults: wrong payload, invalid program, protective stop, controller killed, controller frozen, PolyScope frozen, network cut, CPU starvation, CPU overload, network delay, packet loss, port 30003 blocked, disk full. `wait_ready` brings the robot back to RUNNING. Usable alone: `python -m tools.fault_injection.inject wrong_payload`.

**`capture.py`** 207 lines

Provokes one fault and saves all the evidence: the raw new log lines (read by byte position and filtered by time), fault codes, mode changes, stream cuts and rate, new black boxes, Dashboard state, recovery time. One JSON per run in `data/catalogue/`. `-repeat 5` for five runs.

**`catalogue.py` + `catalogue.yaml` + `catalogue_template.tex`** 234 + 143 + 79 lines

Builds the fault catalogue PDF. The rule: *nothing observed is written by hand.* `catalogue.yaml` holds only the commands and the verified official sources; `catalogue.py` computes the observed columns from the captures (with "5/5"-style run counts) and fills the LaTeX template.

**`netem/Dockerfile`** 4 lines

The Alpine helper image for network faults.

**`scenarios.yaml`** 43 lines

The benchmark scenarios: a fault, the mode (loop or check), and the expected answer (a sentence for the judge, keyword groups).

**`tools/fetch_ur_docs.py`** 83 lines

Downloads the official error code pages into `manuals/ursim/`.

### 9.2 Benchmarks: `benchmarks/run.py`

`run.py` 148 lines

For each scenario: repair the robot, inject the fault, run the agent, grade the root cause twice (keywords, and a Claude judge answering in a strict JSON format with low effort), repair the fault. Prints correct diagnoses, mean time and total cost; saves the results in `benchmarks/results/`.

### 9.3 Tests: `tests/`

24 tests, run with `.venv/bin/python -m pytest`, using the fake arm and a scripted fake Claude (no network, no cost):

- `test_agent.py` (5): a full check produces a report; every movement needs approval; an out-of-range test is refused without moving; loop mode wakes on a fault and asks; loop mode stops if the operator declines.
- `test_core.py` (10): safety bounds and PIN, tool schema, watcher ("default" is not a "fault"), ring buffer, recorder stop, save/load, a stream that stops for good, settings.
- `test_ursim.py` (9): Real-Time and Primary packets, the three log formats, colours, multi-line messages, pendant lines, documentation search, black box reading, Markdown sections.

## 10 How it was developed

### 10.1 The code principles (CONTRIBUTING.md)

1. **Type annotations everywhere**: every parameter and return value says its type.
2. **Docstrings everywhere**: every module, class and function starts with a sentence saying what it does (your choice). This is why about a quarter of the lines are explanations.
3. **Deep modules** (J. Ousterhout, *A Philosophy of Software Design*): a simple interface hiding a rich implementation; no tiny functions called once; each file hides one decision (e.g. only `ursim.py` knows the UR byte formats).
4. **Logging**: every module logs; levels have a meaning; secrets are never logged.
5. **Comments only for the why**, never to paraphrase the code.
6. **English everywhere**; `ruff` and `pytest` pass before every commit.

## 10.2 The history of this branch

| | |
|---|---|
| `3f6230e` | Refactor: software detection only, English, no baselines, no RoArm or physical code, redundancy removed. |
| `149501b` | URSim study: evidence capture, pendant log, black boxes, stream-cut fixes. |
| `22d0ba1` | Official UR error code pages as documentation. |
| `55d2ac9` | Four new faults: PolyScope freeze, network cut, CPU starvation, CPU overload. |
| `d375ff4` | Network and disk faults; URSim moved to its own small disk. |
| `e27d060` | Capture probe for disk full, `-repeat`. |
| `934bbc3` | Catalogue generator (observed columns computed only). |
| `f48d821` | `Recorder.stop()`: fixes the polluted captures. |
| `f6c35d2` | The fault catalogue (13 faults × 5 runs) and the detection ideas. |

## 10.3 Bugs found by testing on the real simulator

Each of these was invisible in the unit tests and only appeared on URSim. They are the reason the study was worth doing before writing detection rules.

- `tail` kept running inside the container after we stopped (fixed in `logs()`).
- A stream that stopped for good was not counted as a cut (fixed in `stream_gaps`).
- Recorders were never stopped, and their leftover connections added cuts and warnings to unrelated faults from round 4 on. Fixed, tested, and all 65 captures redone.
- Log files contain NULs and get rewritten: reading by line number failed; now by byte position and time.
- Killing PolyScope stops the whole container (no supervisor), so that fault became a freeze instead.
- `docker update -cpus 0` does not remove a CPU limit.
- The mean sample rate looked normal for a stream that had stopped; the catalogue uses the effective rate (samples received ÷ window length).

# 11 How big is it?

## 11.1 Lines, by part

| Part | Total | Code | Docstr. | Comm. | Blank |
|---|---|---|---|---|---|
| Agent package (`src/`) | 2 668 | 1 532 | 678 | 28 | 430 |
| Fault tools (`tools/`) | 759 | 492 | 137 | 19 | 111 |
| Benchmarks | 149 | 107 | 21 | 1 | 20 |
| Tests | 389 | 256 | 47 | 0 | 86 |
| **All Python** | **3 965** | **2 387** | **883** | **48** | **647** |

Plus about 820 lines of Markdown, YAML and LaTeX (README, summary, scenarios, catalogue sources and template, system prompt).

**The short answer:** the project is **about 4 000 lines of Python**, of which **about 2 400 are real code**. The agent itself is **2 668 lines in 19 files, 1 532 of real code**. The rest is explanations (docstrings, your choice), blank lines, and the tools that build the fault dataset.

## 11.2 The agent package, file by file

| File | Total | Code |
|---|---|---|
| connectors/ursim.py | 672 | 426 |
| core/recorder.py | 305 | 190 |
| agent/graph.py | 251 | 145 |
| agent/tools.py | 239 | 164 |
| docs.py | 150 | 91 |
| ___main___.py | 141 | 84 |
| agent/llm.py | 141 | 75 |
| connectors/fake.py | 136 | 72 |
| core/events.py | 123 | 49 |
| agent/context.py | 113 | 48 |
| core/checks.py | 87 | 36 |
| config.py | 82 | 48 |
| hmi.py | 77 | 41 |
| core/safety.py | 76 | 38 |
| core/interfaces.py | 69 | 24 |
| 5 `__init__.py` files | 7 | 1 |

### 11.3 Compared with `main`

On `main`, the agent package was 2 917 lines in 33 files. It is now 2 668 lines in 19 files, *while gaining* the pendant log, the black boxes, the official documentation and many fixes. The parts that grew are the fault tools (210 → 759 lines: 13 faults instead of 5, plus capture and catalogue) and the tests (273 → 389).

## 12 What is missing

In order of importance for the project.

### 12.1 1. The detection rules on the signals

`core/checks.py` only watches logs and fault codes. Rules b (target vs reality), e (joint effort ratio), a (hardware limits) and c (recent past) are decided but **not written**, and `config/detection.yaml` does not exist yet. Consequence, measured in the catalogue: in loop mode, **4 of the 13 faults raise no alert at all** (wrong payload, controller frozen, CPU starvation, CPU overload).

### 12.2 2. The expected current in the live stream

Rules b and e need the current the controller *expects*. Today only the black boxes have it (`current_target`); the live Real-Time stream reads the target torque but not the target current. Its position in the packet must be checked on URSim first.

### 12.3 3. A "stream health" rule (proposed rule g)

An alert when fewer samples arrive than expected, or none. It would catch 8 of the 13 faults alone, including 3 that the logs miss. Waiting for your decision.

### 12.4 4. Benchmarks: never run, and incomplete

- **The agent has not yet done a full investigation with the real Claude on this branch**: `data/reports/` and `benchmarks/results/` are empty. Everything in the agent is tested with a fake Claude only.
- `scenarios.yaml` covers only 5 of the 13 faults.
- So the statistics (% detected, % correct cause, detection time, cost) do not exist yet.

### 12.5 5. Smaller gaps in the code

- The summary Claude gets after a test gives the *mean* rate between the first and last sample, which can look normal when the stream stopped (the catalogue already uses the effective rate; the agent should too). The cuts are reported, so it is not hidden, only misleading.
- `Recorder.subscribe()` has no "unsubscribe": each loop scenario of a benchmark adds a queue that keeps filling. Harmless for one run, a slow memory leak over many.
- The documentation is only the error code pages: no User or Service Manual yet, so Claude cannot quote a repair procedure or a hardware limit (which rule a needs).
- No tests for the fault tools, the benchmark runner, the cost computation or the plot/table budget; no automatic end-to-end test against URSim; no continuous integration.
- An old `data/baselines/` folder from before the refactor is still on disk (not in git).

### 12.6 6. Planned for later (summary.md, section 6)

- The RoArm-M2-S connector, with log lines and error codes we generate ourselves.
- Physical anomaly detection: ESP32 → USB → Raspberry Pi, FFT and rules on the Pi, Claude explains.
- Running everything on the Raspberry Pi.
- An interface other than the terminal (web, notifications), and reports in a readable format (today: JSON + terminal).

## 13 Glossary

**URSim**
The official Universal Robots simulator: the real controller software, without the arm.

**URControl**
The controller program: computes movements, serves ports 30001–30004.

**PolyScope**
The robot's user interface (on the tablet). Serves the Dashboard (29999) and writes the black boxes.

**Teach pendant**
The tablet used to program and drive the robot. Its event list is `log_history.txt`.

**URScript**
The robot's programming language (`movej`, `set_target_payload`...).

**Protective stop**
The robot stops because a safety limit was crossed; code `C209` when triggered by a program.

**Robot mode / safety status**
e.g. RUNNING, POWER_OFF / NORMAL, PROTECTIVE_STOP.

**Black box (flight report)**
Zip written by the robot on each error: 30 s of its own data and logs.

**Payload**
The mass declared for the tool. A wrong value makes the expected torques wrong.

**Following error**
The gap between the position the controller asked for and the real one.

**Ring buffer**
A fixed-size memory where new data overwrites the oldest.

**BM25**
The ranking formula of classic search engines.

**Token**
The unit Claude reads and is billed by, about 3–4 characters of text.

**Prompt caching**
Claude's API keeps the start of a conversation in memory, so resending it costs much less (20 times less for Opus 5.5).

**Adaptive thinking / effort**
Claude decides how long to think; "effort" sets how thorough it is.

**LangGraph**
A library to describe a program's flow as a graph of steps.

**netem / iptables**
Linux tools to delay or lose network packets / block a port.

**Docker container**
An isolated mini-system running one program (here URSim) on the laptop.